

Library Management System — Project Report

Programming in Java (Flipped Course) | VITYarthi | Submitted: September 2026

Student: Shamique Khan • Reg. No.: 25BAI10187 • Programme: B.Tech — Artificial Intelligence and Machine Learning • Year: 2026–2027

Project Type: Evaluated Flipped Course Project | **Domain:** Library Management | **Repository:**
<https://github.com/shamiquekhan/Library-Management-System> | **Visibility:** Public

2 Introduction

This project implements a **Library Management System (LMS)** as a Java desktop application developed for the *Programming in Java* flipped course. The default interface is a **Java Swing GUI** (light-themed dashboard with role-aware navigation); the original terminal workflow remains available via `- c\l.i`. The system supports three user roles (Librarian, Clerk, Member), persists all data in an embedded SQLite database through JDBC, and runs two scheduled background services (overdue monitoring and automated backup) using the Concurrency APIs.

Every major topic of the five course modules is applied in a real domain: Java fundamentals and file I/O, object-oriented design, a custom exception hierarchy, the Collections Framework, and JDBC transactions alongside scheduled concurrency — plus the GUI concepts of event handling, layout managers and `SwingWorker` concurrency. The application is built and launched entirely from the command line (`./build.sh`, `./run.sh`) with no external database server and no web/frontend framework, so an evaluator can build and run it immediately after cloning.

3 Problem Statement

Libraries manage a continuous cycle of cataloguing, circulation and accountability: books are added and withdrawn, members join, books are issued and returned, popular titles accumulate hold queues, overdue loans attract fines, and administrators need statistics, reports and safe backups. Managing this cycle by hand is slow and error-prone: due dates are miscounted, fines are calculated inconsistently, hold queues are lost, and no audit trail exists. Spreadsheet-style records share none of the validation, transactional consistency or role-based access that a purpose-built system provides.

This project delivers a digital solution that (1) automates cataloguing, membership, circulation, holds and fine calculation with enforced business rules; (2) provides role-appropriate interfaces for Librarian, Clerk and Member; (3) keeps all records durable in a relational database with transactional updates; and (4) produces reports, statistics, logs and backups without manual effort.

4

Objectives

- Apply Java syntax, control structures, strings, arrays and I/O fundamentals in a realistic domain.
- Design an object-oriented model using inheritance, abstraction, polymorphism, encapsulation, interfaces and enums.
- Implement a custom checked exception hierarchy with validation and error propagation at every layer.
- Use file I/O (`BufferedReader/BufferedWriter`, `PrintWriter`, `Properties`) for configuration, reports, logs, backup manifests and CSV import.
- Select and justify Collections Framework structures (`List`, `Map`, `Set`, `Queue`, `Comparator`) for each data-management need.
- Build a JDBC application over SQLite using `PreparedStatement`, generated keys and explicit transactions.
- Demonstrate concurrency with `ScheduledExecutorService` daemon tasks and `synchronized` service methods.
- Deliver a reproducible automated test suite alongside the manual test evidence.

5

Functional Requirements

The system provides five major functional modules, exceeding the required minimum of three.

ID	Module	Inputs	Processing	Outputs
FR-01	Authentication & Roles	Username, password, role	Credential + role verification; password change with old-password check	Role-specific console; <code>InvalidCredentialsException</code> on failure
FR-02	Book Catalog	ISBN, title, author, subject; keyword; CSV path	CRUD with duplicate-ISBN and status checks; search by title/author/subject; CSV import	Updated catalog, listings, import summary (added/skipped)
FR-03	Circulation	Book ID, member ID	Issue (loan limit 3, fine/overdue gates, hold enforcement), return (fine Rs. 5/day, hold fulfilment via request-date ordered <code>PriorityQueue</code>), renew (max 2, blocked by holds/overdue), place hold (max 2 pending), collect fine	Loans with due dates, fine amounts, hold positions, payment confirmations
FR-04	Reporting & Statistics	Current database records	CSV/TXT export with timestamped names; per-subject aggregation in <code>TreeMap</code>	<code>reports/*_report_<timestamp>.{csv,txt}</code> , on-screen statistics
FR-05	Automated Services & Administration	Configured intervals; member data	Scheduled overdue scans refresh fines; scheduled file backup + manifest; member registration with uniqueness check	Updated fines, <code>backups/library-backup-<timestamp>.db</code> + manifest, activity log entries

6 Non-Functional Requirements

Requirement	Implementation / Evidence
Performance	SQL indexes on <code>loan(member_id)</code> and <code>hold_request(book_id)</code> ; <code>HashMap/TreeMap</code> aggregation; embedded SQLite with zero server overhead
Reliability	JDBC transactions (<code>commit/rollback</code>) around <code>issue/return</code> ; <code>try-with-resources</code> on every stream and connection
Security	Role-separated menus; <code>PreparedStatement</code> exclusively (no SQL concatenation); plaintext passwords documented as a limitation
Usability	Numbered menus, range-validated input with re-prompt, blank-keeps-current updates, consistent <code>[ok]/[x]</code> messages, seeded demo data on first run
Maintainability	Layered packages <code>ui</code> → <code>service</code> → <code>dao</code> ; cross-cutting utilities isolated in <code>lms.util</code> ; each layer depends only on the layer below
Error Handling	Custom checked hierarchy: <code>LMSEException</code> + 10 subclasses; exception chaining via <code>initCause</code> ; multi-catch in CSV import; UI prints messages without stack traces
Logging & Monitoring	Timestamped activity log (<code>logs/lms.log</code>) for logins, circulation, holds, fines, imports, backups and monitor scans; console echo
Resource Efficiency	Both background services are daemon threads on a single-threaded <code>ScheduledExecutorService</code> ; JVM exits cleanly

7 System Architecture

Three-layer architecture with cross-cutting services. Each layer communicates only with the layer beneath it: the UI catches `LMSEException` and prints messages, the service layer owns all business rules, and the DAO layer owns all SQL.

```

Console UI (ConsoleUI, LibrarianConsole, ClerkConsole, MemberConsole)
    ↓
LibraryService (business rules, validation, synchronized operations)
    ↓
DAO layer (BookDao, PersonDao, LoanDao, HoldDao – JDBC / PreparedStatement)
    ↓
SQLite database (embedded file, PRAGMA foreign_keys=ON)

Cross-cutting: Config, ActivityLog, ReportGenerator,
               OverdueMonitor, BackupService, FineCalculator, DateTimeUtil

```

PACKAGE STRUCTURE (41 JAVA SOURCE FILES)

Package	Files	Contents
<code>lms.model</code>	10	Person hierarchy, Book, Loan, HoldRequest, BookStatus, Displayable
<code>lms.exception</code>	11	<code>LMSEException</code> + 10 subclasses

Package	Files	Contents
lms.dao	4	BookDao, PersonDao, LoanDao, HoldDao
lms.service	1	LibraryService
lms.reports	1	ReportGenerator
lms.concurrency	2	OverdueMonitor, BackupService
lms.util	7	Console, Config, Database, DateTimeUtil, FineCalculator, ActivityLog, TablePrinter
lms.ui	4	ConsoleUI, LibrarianConsole, ClerkConsole, MemberConsole
lms	1	Main entry point

8

Design Diagrams

Rendered versions are maintained in `design/*.md` and `images/` in the repository; the diagrams are summarised here.

8.1 USE CASE DIAGRAM

Actors: Librarian, Clerk, Member

Librarian: Add/Update/Remove Book, Import CSV, Register Member,
View Members/Loans/Overdue, Subject Statistics,
Generate Reports, Backup Database, Change Password

Clerk: Issue Book, Return Book, Renew Loan, Collect Fine,
Register Member, Search/View Books, View Member Loans
& Fines, Update Member Contact, Change Password

Member: Search/View Books, View My Loans, Place Hold,
View My Holds, View My Fines, Change Password

All actors authenticate (Login). Clerk and Member share catalog search. Return Book «extends» Hold Fulfilment: when a returned book has pending holds, the earliest hold is fulfilled and the book is reserved at the desk.

8.2 WORKFLOW DIAGRAM

Start → Login → role?

- └ Librarian → manage catalog/members → reports/backup → logout
- └ Clerk → issue ≠ return → renew → collect fine → logout
- └ Member → search → place hold → view loans/fines → logout

Return branch: overdue? → calculate fine → record unpaid fine

holds pending? → fulfil earliest hold → RESERVED

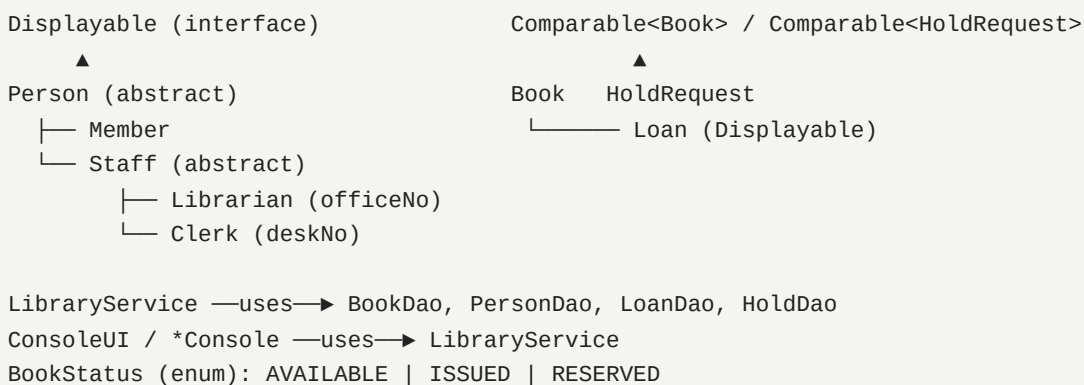
otherwise → AVAILABLE

8.3 SEQUENCE DIAGRAM — ISSUE BOOK (PRIMARY FLOW)

Clerk → ConsoleUI → LibraryService → BookDao/PersonDao/LoanDao/HoldDao → SQLite

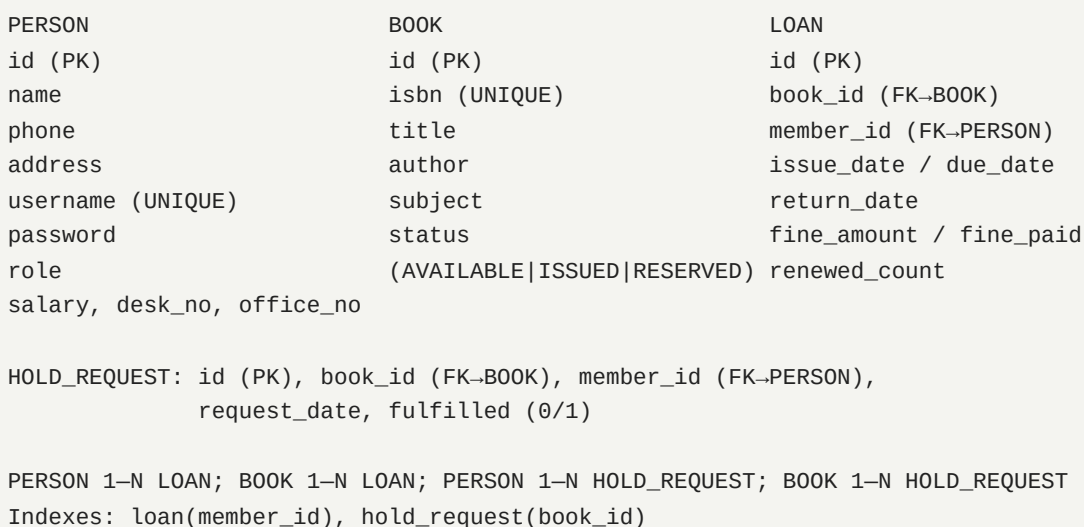
1. findById(book), findById(member) — existence checks
2. Validations: status, hold queue ownership, loan limit, duplicate loan, unpaid fines, overdue loans
3. BEGIN (setAutoCommit false)
4. INSERT loan → UPDATE book ISSUED → [mark hold fulfilled]
5. commit() — on any SQLException: rollback()
6. "[ok] Issued. Due on ..."

8.4 CLASS DIAGRAM



display()/toString() overrides give polymorphic console rendering; Book orders by title, HoldRequest by request date.

8.5 ER DIAGRAM



Note: This schema matches the DDL in `lms.util.Database` exactly. Single-table inheritance is used for person (role discriminator with nullable salary/desk/office columns).

9

Design Decisions & Rationale

Decision	Rationale
Console-based UI	Evaluation requires command-line executability; menu-driven CLI is fully testable via scripted input
SQLite (embedded)	Zero-config file database; evaluator runs the project with no server installation
Plain JDBC	Direct application of Module 5; no ORM hides the SQL being demonstrated
PreparedStatement everywhere	Injection-safe parameterised queries and correct JDBC practice
DAO pattern	Separates persistence from business rules; each DAO owns its table's SQL
Service layer	Centralises all business rules (limits, fines, holds) in one auditable class
Explicit transactions	Issue/return touch multiple tables; commit/rollback guarantees consistency
PriorityQueue<HoldRequest> ordered by request date	First-requested-first-served hold fulfilment via natural ordering
ScheduledExecutorService daemons	Meaningful concurrency tasks (overdue fines, backups) rather than artificial loops
synchronized service methods	Thread-safe mutations against background monitor threads
Custom checked exceptions	Domain failures handled explicitly; causes chained for diagnosis
Properties configuration	Loan period, fine rate, intervals and directories tunable without recompilation
.gitignore for runtime dirs	data/, logs/, backups/, reports/, build/ stay untracked; JDBC JAR is tracked

10

Course Module Mapping

Module	Course Topics	Evidence in Project
Module 1	Java basics, packages, control statements, arrays, strings, I/O fundamentals	Main, Console, Config, TablePrinter; switch, do-while, enhanced for, String.format, Scanner
Module 2	Classes, objects, inheritance, abstraction, polymorphism, encapsulation, interfaces, enums	Person → Member / Staff → Librarian / Clerk; Displayable; BookStatus enum; Comparable on Book/HoldRequest; overridden toString/display()
Module 3	Exception handling (try/catch/finally, multi-catch, throws, custom exceptions), I/O (streams, readers/writers, Properties)	LMSEException + 10 subclasses; try-with-resources on JDBC and files; chained exceptions (initCause); Properties load/store; BufferedReader/BufferedWriter, PrintWriter; CSV parsing

Module	Course Topics	Evidence in Project
Module 4	Collections: List, Map, Set, Queue, Comparator, utility classes	<code>ArrayList</code> (listings), <code>HashMap</code> (stats accumulation), <code>TreeMap</code> (sorted stats), <code>TreeSet</code> (distinct subjects), <code>PriorityQueue<HoldRequest></code> ordered by request date, <code>LinkedList</code> (activity ring buffer), <code>Collections.sort + Comparator</code>
Module 5	JDBC (DriverManager, Connection, PreparedStatement, ResultSet, transactions), concurrency (ExecutorService, synchronized, daemon threads)	<code>Database.open()</code> , <code>PreparedStatement</code> exclusively, <code>setAutoCommit(false)</code> transactions, generated keys; <code>OverdueMonitor</code> + <code>BackupService</code> as <code>ScheduledExecutorService</code> daemons; <code>synchronized</code> service methods

11

Implementation Details

11.1 OBJECT-ORIENTED DESIGN (MODULE 2)

`Person` and `Staff` are abstract; `Member`, `Librarian` and `Clerk` are concrete. All fields are private with accessors. `Book`, `Loan` and `HoldRequest` implement `Displayable` for uniform rendering; `Book` and `HoldRequest` implement `Comparable`. `BookStatus` is an enum guarding all state transitions.

11.2 EXCEPTION HANDLING (MODULE 3)

`LMSEException` is the checked root with 10 subclasses. Service methods declare specific `throws`. `OperationFailedException` wraps `SQLException` with `initCause`. `importBooks()` uses multi-catch (`IOException` | `SQLException`). The UI catches `LMSEException` per flow and prints the message.

11.3 FILE I/O (MODULE 3)

`Config` loads/stores `config/app.properties` (defaults written on first run). `ReportGenerator` writes timestamped CSV/TXT reports via `BufferedWriter`. `ActivityLog` appends to `logs/lms.log`. `BackupService` copies the DB file with `Files.copy` and appends the manifest via `PrintWriter`. CSV import reads via `BufferedReader`, skipping headers, blanks and duplicate ISBNs.

11.4 COLLECTIONS FRAMEWORK (MODULE 4)

`ArrayList` for ordered listings; `HashMap` for $O(1)$ statistic accumulation; `TreeMap` for sorted output; `TreeSet` for distinct subjects; `PriorityQueue` (natural ordering by request date) to select the earliest pending hold on return; `LinkedList` as a bounded recent-activity ring buffer (max 50); `Comparator/Collections.sort` for view-level ordering.

11.5 JDBC & TRANSACTIONS (MODULE 5)

Driver loaded with `Class.forName("org.sqlite.JDBC")` (friendly error if the JAR is missing). Every query uses `PreparedStatement`; inserts retrieve generated keys. Issue and return run inside explicit transactions:

```
c.setAutoCommit(false);
try {
    loanDao.insert(c, loan);
    bookDao.updateStatus(c, bookId, BookStatus.ISSUED);
    if (consumedHold != null) holdDao.markFulfilled(c, consumedHold.getId());
    c.commit();
} catch (SQLException e) {
    c.rollback();
    throw new OperationFailedException("Issue failed, changes rolled back.", e);
} finally {
    c.setAutoCommit(true);
}
```

11.6 CONCURRENCY (MODULE 5)

`OverdueMonitor` runs every `overdue.check.seconds` (default 60), refreshing fines and logging the scan. `BackupService` runs every `backup.interval.seconds` (default 300), copying the SQLite file and updating the manifest. Both use a single-threaded `ScheduledExecutorService` with daemon threads. Mutating `LibraryService` methods are synchronized so monitor scans and console operations never interleave unsafely.

12

Screenshots / Results

The application offers two interfaces — a Swing desktop GUI (default) and a terminal UI (`--cli`); results are captured as console transcripts and GUI screenshots (`screenshots/gui/`). Verified extracts:

LIBRARIAN STATISTICS

```
Enter choice: 10
Books per subject:
ARTIFICIAL INTELLIGENCE   1
DATABASES                 2
...
Total books: 8
```

CIRCULATION WITH FINE AND HOLD FULFILMENT

```
Enter choice: 2          (Return a Book)
Book ID: 5
[ok] Returned with fine Rs. 55.00 (collect at 'Collect Fine').
[log] Book 5 returned by member 3 with fine Rs. 55.00
[log] Hold fulfilled for member 4 – book reserved at desk.

Enter choice: 4          (Collect Fine)
Member ID: 3
[ok] Collected Rs. 55.00.
```

AUTOMATED SMOKE TEST (REPRODUCIBLE)

```
$ ./tests/run_smoke_test.sh
[PASS] TC01 Valid librarian login
[PASS] TC05 Overdue return calculates fine
...
Result: 86 passed, 0 failed
ALL TESTS PASSED
```

13

Testing Approach

Testing is two-layered: a reproducible automated smoke test plus documented manual scenarios.

13.1 AUTOMATED SMOKE TEST

`tests/run_smoke_test.sh` runs the compiled application in a temporary sandbox with a fresh database, feeds a deterministic scripted session covering all three roles, and asserts exact expected outputs (65 checks). It exits 0 only when every assertion passes; on failure the raw console output is preserved for diagnosis.

`tests/test_cases.md` documents the matrix; `tests/expected_output.txt` holds a verified passing run.

13.2 TEST MATRIX (EXECUTED 2026-09-17, OPENJDK 25.0.3, LINUX)

ID	Input / Action	Expected Result	Status
TC01	Valid librarian login	Librarian menu opens	PASS
TC02	Clerk login, wrong password	Incorrect password.	PASS
TC03	Role-mismatch login	Role-mismatch rejection	PASS
TC04	Issue available book	Loan created, due +14 days	PASS
TC05	Return book 11 days overdue	Fine Rs. 55.00 (11 × Rs. 5)	PASS
TC05	Return on time	No fine	PASS
TC06	Issue already-issued book	Rejected (issued to another member)	PASS
TC06	Issue at loan limit (3)	Loan limit reached (3 books).	PASS
TC07	Issue while fine pending	Pending fine of Rs. 55.00	PASS
TC08	Collect fine	Collected Rs. 55.00.	PASS
TC10	Member views loans	2 active loans listed	PASS
TC11	Hold on issued book	Queued, position 1	PASS
TC11	Reserved book issued to first hold owner	Hold consumed, loan created	PASS
TC12	Hold own borrowed book	Rejected	PASS

ID	Input / Action	Expected Result	Status
TC13	Hold available book / third hold per member	Rejected (borrow directly / hold limit)	PASS
TC14	Renew active loan / third renewal / renewal with pending hold	New due date / limit reached / renewal blocked	PASS
TC15	Subject statistics	Sorted counts, total 8	PASS
TC16	View all books	8 seeded books listed	PASS
TC17	Add book	Book added with ID 9.	PASS
TC18	Duplicate ISBN	ISBN already exists: 978-TEST-0001	PASS
TC19	Double return	This book is not currently issued.	PASS
TC20	Books CSV report	File created in reports/	PASS
TC21	CSV import (6 rows)	6 book(s) imported.	PASS
TC22	Backup now	Timestamped backup in backups/	PASS
TC23	Runtime artefacts	data/library.db, logs/lms.log created	PASS
TC24	Dashboard overview (KPIs, charts)	15 books, 3 active loans, status/subject charts	PASS
TC24	Circulation / Members / Hold Queue drill-downs	Attention table, member loans/fines, grouped holds	PASS
TC25	Member: My Dashboard	Own loans + fines KPIs, MY LOANS table	PASS
—	Full session	No unhandled exceptions; clean exit	PASS

13.3 MANUAL SCENARIOS ADDITIONALLY VERIFIED

Negative-input handling of `Console.readInt` (re-prompt); delete blocked for issued/reserved books; CSV import skipping duplicate ISBNs with reported count; overdue monitor scan entries in `logs/lms.log`; backup manifest append; password change followed by re-login with the new password.

14

Challenges Faced

Challenge	Solution
Maintaining database consistency during issue/return (multi-table updates)	Explicit JDBC transactions with commit/rollback so partial updates are impossible
Preventing invalid library operations where many rules interact	All rules centralised in <code>LibraryService</code> ; every failure expressed as a specific custom exception
Running periodic tasks without blocking the CLI	<code>ScheduledExecutorService</code> with daemon threads; synchronized service methods for shared-state safety
Keeping UI, business logic and persistence decoupled	Layered <code>ui</code> → <code>service</code> → <code>dao</code> structure with model classes shared via <code>Displayable</code>

Challenge	Solution
Making testing reproducible rather than claim-based	Sandboxed scripted smoke test with 65 assertions and a machine-checkable exit code, plus an in-process GUI self-test that drives the Swing app and captures screenshots
Keeping runtime artefacts out of the repository	Configurable output directories and <code>.gitignore</code> rules; bundled JDBC JAR tracked

15

Learnings & Key Takeaways

- Object-oriented design pays off across a whole application: the `Person` hierarchy and `Displayable` interface let the UI treat all model objects uniformly.
- JDBC is more than queries — transactions (`setAutoCommit`, `commit`, `rollback`) are what make multi-table operations trustworthy.
- Choosing the right collection matters: `PriorityQueue` ordered by request date expresses hold fulfilment directly; `TreeMap` yields sorted statistics for free.
- A custom checked exception hierarchy produces far clearer errors than blanket `catch (Exception)`, while chaining preserves the root cause.
- `ScheduledExecutorService` with daemon threads is a clean way to run background work in a CLI application that must exit promptly.
- Layered packaging (`ui` → `service` → `dao`) made the automated test suite straightforward to build.
- Configuration via `Properties` separated behaviour from code and enabled sandboxed test runs.
- A reproducible test script converts "it worked when I tried it" into evidence anyone can re-run in seconds.
- Git and GitHub — with meaningful commits and ignored runtime artefacts — are part of deliverable quality.

16

Limitations

- **Plaintext passwords** — acceptable for this educational scope, unsuitable for production.
- **Single-branch library** — no multi-branch support or `branch_id` dimension.
- **No reservation expiry** — a fulfilled hold waits at the desk indefinitely.
- **No notifications** — members learn about hold fulfilment or fines only at the console.
- **File-copy backups** — a copy taken mid-transaction could be momentarily inconsistent; `VACUUM INTO` would guarantee consistency.
- **Console-only UI** — intentional per evaluation requirements.

17

Future Enhancements

- Hash passwords with PBKDF2/BCrypt and add session tokens.
- Multi-branch support with branch-level catalog separation.
- Email/SMS notifications for overdue reminders and hold fulfilment.
- Reservation expiry with automatic release of uncollected holds.
- Consistent hot backups via SQLite `VACUUM INTO`.

- A unit-test layer (JUnit) complementing the end-to-end smoke suite.
- Optional REST API layer for integration beyond the console.

18 Conclusion

This Library Management System demonstrates an integrated application of all five modules of the *Programming in Java* course in one coherent, runnable product: language fundamentals and I/O, object-oriented design, a disciplined exception hierarchy, purposeful use of the Collections Framework, and JDBC persistence with real transactions alongside scheduled concurrency. Beyond features, the project emphasises engineering discipline — a layered architecture, a reproducible 41-assertion test suite, honest documentation of limitations, and clean separation of runtime artefacts from source. The code is original work written specifically for this evaluation.

19 References

1. VITYarthi — *Programming in Java* course materials, Modules 1–5.
2. Oracle — The Java™ Tutorials: Collections, JDBC Database Access, Concurrency.
3. Xerial — `sqlite-jdbc` 3.53.4.0 driver documentation.
4. Java SE API documentation — `java.time`, `java.sql`, `java.util.concurrent`.
5. SQLite documentation — file format, locking and `VACUUM INTO`.

Project Title: Library Management System — Programming in Java (Evaluated Project)

Student: Shamique Khan | **Registration No.:** 25BAI10187 | **Course:** Programming in Java — VITYarthi Flipped Course

Repository: <https://github.com/shamiquekhan/Library-Management-System> | **Visibility:** Public